## Introduction

In today's fast-paced, interconnected world, systems are increasingly required to perform flawlessly, even in the face of failure. Imagine a high-speed train making its way through a busy city. To ensure the train runs smoothly, there are numerous backup systems in place like backup power, maintenance teams, signaling systems, and fail-safe measures. Even if one component fails, the train continues running smoothly, thanks to these safety nets. This analogy of the high-speed train perfectly illustrates the essence of resilience in systems.

In the same way, modern software systems, networks, and infrastructures must be designed to withstand failures and continue functioning seamlessly. Whether it's handling a sudden surge in traffic, recovering from unexpected outages, or ensuring data is always available, resilience plays a pivotal role in maintaining system reliability and user trust.

Understanding how to build resilient systems is essential for anyone involved in system architecture, software development, or infrastructure management. The goal is to design systems that not only deliver exceptional performance but also recover gracefully when failure strikes. In a world where system downtime can result in significant financial and reputational loss, building resilient systems has become a top priority.

Before we dive deeper into the specific techniques and strategies for building resilient systems, let's first define a few key terminologies to ensure we're all on the same page.

---

## Key Terminologies

**Error Handling**

Error handling is the process of anticipating, detecting, and resolving issues during system execution. It ensures that a system responds to failure gracefully without corrupting data or collapsing the user experience. Effective error handling includes mechanisms for capturing errors, logging them, and providing meaningful feedback to users or system administrators.

In resilient systems, error handling goes beyond merely catching exceptions; it’s about maintaining the system’s stability and performance even when something goes wrong.

---

**Resilience**

Resilience refers to the system’s ability to absorb failure and continue operating. Robust systems recover from failure, while brittle systems crash. Resilience is not about avoiding failure but about surviving it and ensuring minimal disruption to service. A resilient system is designed to handle unexpected events, recover quickly from failures, and maintain a consistent user experience.

Resilience is crucial in systems where uptime and continuous service are vital, such as in financial services or healthcare applications.

---

## Robust System vs. Brittle System

When building resilient systems, it's crucial to understand the difference between **robust** and **brittle systems**.

A **robust system** can gracefully degrade in performance or continue working with limited functionality even if part of the system fails. A **brittle system**, on the other hand, is prone to complete failure when faced with issues or when one component goes down.

Let's now understand the difference between the two systems:

| Characteristic | Robust System | Brittle System |
|---|---|---|
| **System Behavior** | Continues on demand gracefully. | Crashes or freezes. |
| **Error Handling** | Shows cached content or provides degraded service. | Entire system halts on error. |
| **Example** | Netflix showing cached content when the service is down. | Amazon checkout page crashes if payment service is down. |
| **User Experience** | Minimizes disruption to the user experience. | User experience is severely disrupted. |
| **Recovery from Failures** | Can recover or continue with limited functionality. | Fails completely with no fallback. |

| | | |
|---|---|---|
| **Example Scenario** | Amazon checkout page hides recommendation services when the checkout fails. | Amazon checkout fails when payment service is down. |

**Explanation with Real-World Example:**

Imagine you're shopping on **Amazon**. When you try to purchase an item, the **Amazon checkout page** is a critical part of the transaction. If this page is part of a **brittle system**, and the payment service goes down, the entire checkout process may fail, causing the user to abandon the purchase and leading to a negative experience.

However, if **Amazon's checkout page** is designed as a **robust system**, even if the payment service fails, the system could still allow the user to complete the purchase by hiding the recommendation service or by offering the ability to retry. This ensures that the user can still proceed without facing a complete breakdown, demonstrating how a robust system maintains functionality under partial failure.

---

## Graceful Degradation Strategies

#### 1. Return Cached Data

When a live service or an external API fails, the system can fall back on cached data to provide a seamless experience to the user. This strategy ensures that even if the primary service is unavailable, the system continues functioning using stored, potentially outdated, data.

##### Code

In the provided code snippet, we have a `RecommendationService` class that fetches user recommendations. Here's how it works:

```
class RecommendationService {
    public List<String> getRecommendedItems(String userId) {
        try {
            // Attempt to fetch live recommendations
            return recommendationService.fetchLiveRecommendations(userId);
        } catch (Exception ex) {
            // If the live service fails, log the error and fall back to cache
            log.warn("Live service failed, falling back to cache");
            return cacheService.getCachedRecommendations(userId);  // Fallback to cached data
        }
    }

    public List<String> fetchLiveRecommendations(String userId) {
        return List.of("movie-1", "movie-2");  // Simulated live recommendation data
    }
}
```

In this example:

- The `getRecommendedItems` method first tries to get the data from the live service by calling `fetchLiveRecommendations`.
- If the live service fails (e.g., due to a network issue or server downtime), the system catches the exception and logs the failure.
- It then returns the cached recommendations using the `cacheService.getCachedRecommendations` method, ensuring that the user still gets some content, albeit potentially out-of-date.

This method of graceful degradation keeps the user experience intact by providing fallback data when the system is unable to perform as expected.

This strategy works best when the data doesn't need to be real-time and can tolerate being stale for short periods, such as user recommendations, product details, or news articles that don't change frequently.

---

#### 2. Show Fallback UI

When a system or service becomes unavailable, it's crucial to ensure that users still have a meaningful experience. One of the ways to handle this is by **showing a fallback UI**.

This could be a message, a static version of the content, or a default view that informs the user of the issue and provides a way forward (e.g., "try again later").

##### Code

Here's how the code for **showing fallback UI** works:

// Show fallback UI

```
public Menu getMenu(String restaurantId) {
   try {
      // Attempt to fetch the live menu
      return menuService.fetchMenu(restaurantId);
   } catch (Exception e) {
      // If the live menu is unavailable, show a fallback message in the UI
      return new Menu("Menu currently unavailable. Please try again later.");
   }
}
```

In this example:

- The `getMenu` method tries to fetch the live menu from the `menuService.fetchMenu` method.
- If the service fails (e.g., the menu is unavailable due to a server issue), the system catches the exception and instead returns a **fallback UI** in the form of a `Menu` object with a message: **"Menu currently unavailable. Please try again later."**

This strategy ensures that users are not left staring at an empty screen or a broken interface. Instead, they are informed that the content they seek is temporarily unavailable, and they can try again later.

This approach is effective for **user-facing applications** where it's essential to maintain a friendly and informative experience, even during downtime.

Examples of fallback UI include error messages, loading spinners, or simplified versions of the content.

---

#### 3. Queue Requests

Another graceful degradation strategy is **queuing requests**. When a service is temporarily unavailable or experiences high traffic, it’s crucial to prevent the system from becoming overloaded or failing outright. By queuing requests, the system can defer the action until the service is ready to process it, ensuring that no requests are lost, and users are not impacted by service interruptions.

##### Code

```
// Queue request's

public void placeOrder(Order order) {

   try {

      // Attempt to charge the payment

      paymentService.charge(order);

   } catch (Exception e) {

      // If payment fails, queue the order for retry

      orderRetryQueue.enqueue(order);

      log.warn("Payment failed. Queued for retry.");

   }

}
```

In this example:

- The `placeOrder` method attempts to charge the payment for an order.
- If the payment service fails (e.g., due to network issues), the exception is caught, and the order is placed in a retry queue (orderRetryQueue.enqueue(order)).
- A warning is logged to indicate that the payment has failed and the order has been queued for retry.

This ensures that, rather than rejecting the request immediately, the system can retry processing the payment later when the service is available again.

The **queueing requests** strategy is ideal for handling intermittent failures in high-demand systems. It allows for **non-disruptive operations**, ensuring that operations are completed when the system can handle them, without burdening the user with error messages or failed transactions.

---

## Retry Mechanisms

In systems with unreliable or intermittent services, retry mechanisms are essential for improving the user experience during temporary failures. Instead of failing immediately or frustrating users with error messages, a retry mechanism allows the system to attempt the same operation a few times, increasing the chances of success.

There are various retry strategies, each suited to different types of failures and service conditions. Let's explore two of the most common strategies:

**1. Naive Retry**

The **naive retry mechanism** simply retries an operation a fixed number of times when it fails. It works well when the service is expected to recover quickly, but it can lead to excessive load on the system if not handled carefully.

Here's an example of the naive retry mechanism:

```
// Naive retry example
public String getETA() {
   int retries = 3;  // Maximum retry attempts
   while (retries-- > 0) {
      try {
         return etaService.getETA();  // Attempt to fetch ETA from service
      } catch (Exception e) {
         log.warn("Retrying ETA, attempts left: " + retries);
      }
   }
   return "ETA unavailable";  // Return message if all retries fail
}
```

In this example:

- The system tries to fetch the ETA from the `etaService` up to three times.
- If the service fails (due to an exception), the system retries the request until the number of retries is exhausted.
- If all attempts fail, it returns a fallback message **"ETA unavailable"**.

The naive retry mechanism is simple to implement but doesn't account for the possibility of repeated failures, and it can overwhelm the system if retries are not well-managed.

---

**2. Backoff Strategy**

A more sophisticated approach is the **backoff strategy**, which adds a delay between retries, helping to reduce the load on the system and give the service time to recover. A common variation is **exponential backoff**, where the delay increases after each retry attempt.

Here's an example of the **backoff strategy**:

```
// Backoff strategy
public String getETAWithBackoff() throws InterruptedException {
    int retries = 3;
    int delay = 1000; // Initial delay is 1 second
    while (retries-- > 0) {
        try {
            return etaService.getETA();  // Attempt to fetch ETA from service
```

```
        } catch (Exception e) {
            Thread.sleep(delay);  // Wait before retrying
            delay *= 2;  // Exponential backoff: double the delay each time
        }
    }
    return "ETA unavailable";  // Return message if all retries fail
}
```

In this example:

- After each failed attempt, the system waits for a progressively longer time before retrying.
- Initially, it waits for 1 second, then 2 seconds, then 4 seconds, and so on. This **exponential backoff** helps in reducing the strain on the system and avoids flooding it with too many requests.

This approach is useful in scenarios where the service is temporarily overwhelmed, giving it a chance to recover between retry attempts.

---

## Possibility of DDoS due to Retry Mechanisms

**What is DDoS?**

**DDoS (Distributed Denial of Service)** is a type of cyberattack where multiple systems, often compromised and controlled remotely, are used to flood a

target server or network with traffic. The overwhelming volume of requests causes the target system to slow down or crash, denying service to legitimate users.

---

#### How Retry Mechanisms Can Lead to DDoS

While retry mechanisms are crucial for improving resilience, improper implementation can **lead to a self-inflicted DDoS** on your system. Consider this scenario:

- **If many users are retrying failed requests at the same time**, especially with short or no backoff periods, it can cause a significant spike in requests, overwhelming the system.
- This sudden surge in retries can lead to resource exhaustion, slower response times, and potentially system crashes, much like a DDoS attack.

---

#### Possible Solutions

To avoid DDoS-like situations while using retry mechanisms:

- **Implement exponential backoff** (as shown in the backoff strategy) to progressively delay retries, preventing all users from retrying at once.
- **Cap the number of retries** and set a **maximum delay** to prevent endless attempts in case of service failure.
- **Use circuit breakers** to detect persistent failures and stop retrying temporarily, allowing the system to recover instead of continually hammering it with requests.

- **Rate limit** retries from clients to ensure that not too many retries happen in a short time frame.

By using these strategies, you can reduce the risk of unintentionally overwhelming your system and ensure the service remains available to all users.

---

## Circuit Breaker Pattern

The **Circuit Breaker Pattern** is a design pattern used to protect a system from repeatedly calling a failing service. Instead of constantly trying to invoke a failing service, the circuit breaker “opens” after a certain number of failures, blocking further attempts and allowing the system to continue operating. This prevents the system from overloading the failing service and gives it time to recover.

#### The Problem

What happens if a downstream service, such as a **payment service**, is constantly failing? If the system keeps calling the service, it can lead to wasted resources, further failure, and potentially impact the performance of other components. The **Circuit Breaker Pattern** addresses this by **stopping the calls after a threshold** is met, allowing the system to **wait and retry later**.

#### The Solution

The solution is to implement a circuit breaker that stops sending requests to the failing service after a threshold of failures. Once the service has had time to recover, the circuit breaker enters a "half-open" state to test the service's availability before fully restoring normal operations.

---

**States of Circuit Breaker**

- **Closed:** The service is working normally, and requests are being sent to the service.
- **Open:** The service is failing consistently, and the circuit breaker stops sending requests to the service.
- **Half-Open:** After a defined timeout, the system sends a limited number of test requests to see if the service has recovered. If these requests succeed, the circuit breaker returns to the "Closed" state.

---

**Code Implementation**

Here's how the **Circuit Breaker** pattern can be implemented using **Spring Boot** with **Resilience4j**.

**1. Annotation-based Circuit Breaker Setup:**

The circuit breaker is applied using the `@CircuitBreaker` annotation, which wraps the method with the defined circuit breaker and fallback mechanism.

```
@Service

class PaymentService {


   @CircuitBreaker(name = "paymentService", fallbackMethod =
"paymentFallback")

   public String charge(String userId, double amount) {

      // real payment logic
```

```
        return externalPaymentApi.charge(userId, amount);
    }

    // Fallback method in case of failure
    public String paymentFallback(String userId, double amount, Throwable
t) {
        log.error("Payment Service Down. Fallback triggered.");
        return "PAYMENT_FAILED";
    }
}
```

- `@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")`:
  This annotation applies the circuit breaker to the **charge** method. If it fails, it will trigger the fallback method `paymentFallback()`.
- `paymentFallback()`: This method is invoked if the **charge** method fails, providing a default value and logging the failure.

---

#### 2. Java Config Customization:

You can also configure the circuit breaker programmatically by using a **@Bean** method:

```
@Bean
```

```
public Customizer<CircuitBreakerConfigCustomizer>
paymentCircuitBreakerConfig() {
   return CircuitBreakerConfigCustomizer.of("paymentService", builder ->
builder
      .slidingWindowSize(10)
      .failureRateThreshold(50)
      .waitDurationInOpenState(Duration.ofSeconds(10))
      .permittedNumberOfCallsInHalfOpenState(2)
      .automaticTransitionFromOpenToHalfOpenEnabled(true));
}
```

This allows you to configure the circuit breaker dynamically at runtime.

---

**3. Configuration via `application.yml`:**

You can configure the circuit breaker properties externally in the application.yml file as follows:

```
resilience4j:
  circuitbreaker:
    instances:
```

```
paymentService:
  registerHealthIndicator: true
  slidingWindowSize: 10
  slidingWindowType: COUNT_BASED
  minimumNumberOfCalls: 5
  failureRateThreshold: 50
  waitDurationInOpenState: 10s
  permittedNumberOfCallsInHalfOpenState: 2
  automaticTransitionFromOpenToHalfOpenEnabled: true
```

- `slidingWindowSize:` Defines the number of calls to consider when determining whether the circuit should open.
- `failureRateThreshold:` Defines the failure rate (in percentage) above which the circuit will open.
- `waitDurationInOpenState:` The duration the circuit breaker stays open before transitioning to a half-open state.
- `permittedNumberOfCallsInHalfOpenState:` Number of allowed requests in half-open state before deciding whether to close the circuit.

---

## Failover and Timeout Strategies

In a resilient system, it's not just about detecting failure, it's about how quickly and effectively the system recovers or reroutes around that failure. Two key techniques that play a major role in this are **Timeouts** and

**Failovers**. These strategies help ensure that your system doesn't hang indefinitely and can automatically switch to backup options when necessary.

### 1. Timeout Strategy

A **timeout** is a mechanism used to prevent long waits or hangs when a service becomes unresponsive. Instead of waiting indefinitely for a response, a timeout defines a maximum amount of time to wait before the request is aborted.

#### Why It Matters

- Prevents resource blockage and thread exhaustion.
- Ensures the calling service doesn't get stuck.
- Allows for fallback or retry logic to kick in promptly.

#### Example Use Case:

Imagine your frontend service calls a backend API to fetch user details. If that backend service is hanging (maybe due to a DB lock), your frontend should timeout in 2 seconds rather than wait endlessly, ensuring your UI remains responsive or shows a graceful error.

---

### 2. Failover Strategy

**Failover** is the process of switching to a standby or alternative service when the primary one is unavailable or down. This is a proactive resilience strategy that ensures high availability and continued service delivery even when components fail.

#### Why It Matters

- Enables seamless experience even during service failures.
- Minimizes downtime and disruption.

- Useful in both service-to-service calls and infrastructure (like DB or servers).

**Example Use Case:**

If you have two payment gateways (say Razorpay and Stripe), and Razorpay is down, the system can automatically **failover** to Stripe to complete the transaction without the user even noticing.

By combining Timeouts (to avoid hanging) and Failovers (to reroute the request), systems can maintain a smooth and fault-tolerant user experience even in the face of partial or complete component failures.

---

## Summary - Engineering Checklist

To build a resilient and high-performing system, it's essential to apply various strategies that handle different types of failures, delays, and potential issues. Here's a summary of key engineering strategies and solutions you can apply to ensure your system remains functional under different circumstances:

**1. Temporary Spike**

- **Problem:** Sudden bursts in demand or traffic.
- **Solution:** Use **retry with backoff** to manage the load. This technique helps to handle spikes by spacing out retries with a growing delay, reducing the strain on the system and giving it time to recover.

---

**2. Persistent Failure**

- **Problem:** A service or component that is consistently failing.

- **Solution:** Implement a **Circuit Breaker** to stop calling the failing service after a set threshold. This allows the system to stop wasting resources and enter a recovery mode.

---

#### 3. Third-party Delay

- **Problem:** Delays caused by external services (e.g., APIs, cloud services).
- **Solution:** Use **Timeouts** to avoid waiting too long for responses from third-party services. Setting an appropriate timeout ensures that your system doesn't hang indefinitely and can proceed with a fallback.

---

#### 4. Degraded Experience

- **Problem:** System continues working, but with reduced functionality.
- **Solution:** Implement **Fallback UI on Cache**. When live data is unavailable, use cached data or show a degraded user interface to ensure users still have access to important information.

---

#### 5. Avoid Throttling

- **Problem:** Excessive load on the system or external services due to too many simultaneous requests.
- **Solution:** Use **Own Rate Limiting** to limit the number of requests your system can make to external services within a specific time frame, ensuring that it doesn't overwhelm any single service.

---

**6. Highly Critical Services**

- **Problem:** Critical services must remain operational at all times.
- **Solution:** Implement **Failover Setup** to ensure that if one critical service goes down, another backup service takes over seamlessly, ensuring high availability and minimal disruption.

By following these strategies, you can ensure that your system is robust, resilient, and capable of handling various real-world challenges. Implementing these solutions will help protect your system from downtime, service failures, and poor user experiences, all while maintaining functionality even in the face of issues.